



Mata Kuliah : Pemrograman Web Lanjut (PWL)
Program Studi : D4 – Teknik Informatika
Semester : 4

Kelas : 2A
NIM : 244107020030
Nama : Athaulla Hafizh
Jobsheet Ke- : 10

Laporan Jobsheet 10 - 03

Link Repository : <https://github.com/athfizh/Pemrograman-Web-Lanjut>

Praktikum

Langkah	Jawaban/Deskripsi
1	Rollback Migration <pre>PS C:\laragon\www\Pemrograman-Web-Lanjut\jobsheet10\filament-app> php artisan migrate:rollback</pre> <pre>2026_05_18_015348_create_products_table 18.24ms DONE 2026_03_30_173953_create_posts_table 10.13ms DONE 2026_03_30_173507_create_categories_table 5.84ms DONE 0001_01_01_000002_create_jobs_table 19.67ms DONE 0001_01_01_000001_create_cache_table 11.44ms DONE 0001_01_01_000000_create_users_table 16.91ms DONE</pre> <pre>Schema::create('posts', function (Blueprint \$table) { \$table->id(); \$table->string('title'); \$table->string('slug'); \$table->foreignId('category_id')->constrained('categories')->cascadeOnDelete(); \$table->string('color')->nullable(); \$table->string('image')->nullable(); \$table->text('body')->nullable(); \$table->boolean('published')->default(false); \$table->date('published_at')->nullable(); \$table->timestamps(); });</pre>
2	Membuat Tabel Tags



	<pre>Schema::create('tags', function (Blueprint \$table) { \$table->id(); \$table->string('name'); \$table->timestamps(); }); → Tab to Jump /** * Reverse the migrations. */ public function down(): void { Schema::dropIfExists('tags'); Schema::dropIfExists('posts'); } };</pre>
3	Membuat Pivot Table <pre>Schema::create('tags', function (Blueprint \$table) { \$table->id(); \$table->string('name'); \$table->timestamps(); }); Schema::create('post_tag', function (Blueprint \$table) { \$table->foreignId('post_id')->constrained()->cascadeOnDelete(); \$table->foreignId('tag_id')->constrained()->cascadeOnDelete(); \$table->primary(['post_id', 'tag_id']); }); /** * Reverse the migrations. */ public function down(): void { Schema::dropIfExists('post_tag'); Schema::dropIfExists('tags'); Schema::dropIfExists('posts'); } };</pre>
4	Membuat Resource Tag pada Filament



	<pre>PS C:\laragon\www\Pemrograman-Web-Lanjut\jobsheet10\filament-app> php artisan make:filament-resource Tag The "title attribute" is used to label each record in the UI. You can leave this blank if records do not have a title. What is the title attribute for this model? > name Would you like to generate a read-only view page for the resource? (yes/no) [no] > no Should the configuration be generated from the current database colu mns? (yes/no) [no] > no Does the model use soft-deletes? (yes/no) [no] > no</pre>
5	Membuat Model Tag <pre>class Tag extends Model { 0 references protected \$fillable = ['name']; }</pre>
6	Menambahkan Relationship pada Model Post Form Tag & Table Tag telah melengkapi <i>form</i> dan <i>table</i> untuk <i>TagResource</i>: <pre>class TagForm { public static function configure(Schema \$schema): Schema { return \$schema ->components([TextInput::make('name')]); } }</pre>



```
return $table
    ->columns([
        TextColumn::make('name')
    ])
    ->filters([
```

Redirect URL pada Create & Edit Tag

```
filament-app > app > Filament > Resources > Tags > Pages > CreateTag.php > ...
1  <?php
2
3  namespace App\Filament\Resources\Tags\Pages;
4
5  use App\Filament\Resources\Tags\TagResource;
6  use Filament\Resources\Pages\CreateRecord;
7
8  2 references | 0 implementations
9  class CreateTag extends CreateRecord
10 {
11     0 references
12     protected static string $resource = TagResource::class;
13
14     0 references | 0 overrides
15     protected function getRedirectUrl(): string
16     {
17         return $this->getResource()::getUrl('index');
18     }
19 }
```

```
protected function getRedirectUrl(): string
{
    return $this->getResource()::getUrl('index');
}
```

Pembaruan Model Post (Relasi Many-to-Many)

```
public function tags()
{
    return $this->belongsToMany(Tag::class, 'post_tag');
}
```



7	Menambahkan Relationship pada Model Tags <pre>public function posts() { return \$this->belongsToMany(Post::class, 'post_tag'); }</pre>
8	Menggunakan Relationship pada Form Post <pre>// Section C: Meta Information – icon tag Section::make('Meta Information') ->icon(Heroicon::OutlinedTag) ->schema([// TagsInput::make('tags') // ->placeholder('New tag'), Select::make('tags') ->relationship('tags', 'name') ->multiple(), Checkbox::make('published'), DateTimePicker::make('published_at'),]),</pre>
9	Membuat Relationship Manager <pre>Select::make('tags') ->relationship('tags', 'name') ->multiple() ->preload(),</pre>
10	Menghubungkan Relationship Manager <pre>use App\Filament\Resources\Posts\RelationManagers\TagsRelationManager; public static function getRelations(): array { return [TagsRelationManager::class,]; }</pre>
11	Fitur Relationship Manager



```
public function table(Table $table): Table
{
    return $table
        ->recordTitleAttribute('name')
        ->columns([
            TableColumn::make('id'),
            TableColumn::make('name')
            // ->searchable(),
        ])
        ->filters([
            //
        ])
        ->headerActions([
            CreateAction::make(),
            AttachAction::make(),
        ])
        ->recordActions([
            EditAction::make(),
            DetachAction::make(),
            DeleteAction::make(),
        ])
        ->toolbarActions([
            // BulkActionGroup::make([
            //     DetachBulkAction::make(),
            //     DeleteBulkAction::make(),
            // ]),
        ]);
}
```

Latihan Praktikum

Langkah	Jawaban/Deskripsi
1	Buat tabel tags dan pivot table post_tag



	<pre>public function up(): void { Schema::create('posts', function (Blueprint \$table) { \$table->id(); \$table->string('title'); \$table->string('slug'); \$table->foreignId('category_id')->constrained('categories')->cascadeOnDelete(); \$table->string('color')->nullable(); \$table->string('image')->nullable(); \$table->text('body')->nullable(); \$table->boolean('published')->default(false); \$table->date('published_at')->nullable(); \$table->timestamps(); }); Schema::create('tags', function (Blueprint \$table) { \$table->id(); \$table->string('name'); \$table->timestamps(); }); Schema::create('post_tag', function (Blueprint \$table) { \$table->foreignId('post_id')->constrained()->cascadeOnDelete(); \$table->foreignId('tag_id')->constrained()->cascadeOnDelete(); \$table->primary(['post_id', 'tag_id']); }); }</pre>
2	Buat model Tag <pre>0 references 0 implementations class Tag extends Model { 0 references protected \$fillable = ['name']; 0 references 0 overrides public function posts() { return \$this->belongsToMany(Post::class, 'post_tag'); } }</pre>
3	Tambahkan relationship pada model Post



	<pre>public function tags() { return \$this->belongsToMany(Tag::class, 'post_tag'); } public function posts() { return \$this->belongsToMany(Post::class, 'post_tag'); }</pre>
4	Buat Tag Resource <pre>return \$schema ->components([TextInput::make('name')]); return \$table ->columns([TextColumn::make('name')]) ->filters([//])</pre>
5	Tambahkan multiple select tags pada Post form <pre>// Section C: Meta Information – icon tag Section::make('Meta Information') ->icon(Heroicon::OutlinedTag) ->schema([// TagsInput::make('tags') // ->placeholder('New tag'), Select::make('tags') ->relationship('tags', 'name') ->multiple() ->preload(), Checkbox::make('published'), DateTimePicker::make('published_at'),]),</pre>
6	Implementasikan Tags Relationship Manager



```
public function table(Table $table): Table
{
    return $table
        ->recordTitleAttribute('name')
        ->columns([
            TextColumn::make('id'),
            TextColumn::make('name')
            // ->searchable(),
        ])
        ->filters([
            //
        ])
        ->headerActions([
            CreateAction::make(),
            AttachAction::make(),
        ])
        ->recordActions([
            EditAction::make(),
            DetachAction::make(),
            DeleteAction::make(),
        ])
        ->toolbarActions([
            // BulkActionGroup::make([
            //     DetachBulkAction::make(),
            //     DeleteBulkAction::make(),
            // ]),
        ]);
}
```

Analisis dan Diskusi

1. Apa perbedaan HasMany dan Many-to-Many?

- HasMany (One-to-Many): Relasi di mana satu data di tabel A bisa memiliki banyak data di tabel B, tetapi satu data di tabel B hanya dimiliki oleh satu data di tabel A. Penyimpanan relasinya cukup menggunakan satu Foreign Key di tabel B. (Contoh: 1 Kategori memiliki banyak Post).
- Many-to-Many: Relasi di mana satu data di tabel A bisa memiliki banyak data di tabel B, dan sebaliknya satu data di tabel B juga bisa dimiliki oleh banyak data di tabel A. Relasi ini



membutuhkan bantuan tabel ketiga (pivot table). (Contoh: 1 Post bisa punya banyak Tag, dan 1 Tag bisa dipakai di banyak Post).

2. Mengapa pivot table diperlukan?

Dalam prinsip relasional database (RDBMS), sebuah kolom tidak boleh menyimpan lebih dari satu nilai (atomic value) untuk mematuhi kaidah Normalisasi (1NF). Karena relasi Many-to-Many melibatkan banyak nilai dari kedua sisi (Banyak Post dan Banyak Tag), kita tidak bisa menyimpan banyak ID langsung di dalam tabel asal. Oleh karena itu, diperlukan tabel perantara (Pivot Table seperti `post_tag`) yang fungsinya semata-mata untuk menyimpan pemetaan pasangan ID dari kedua tabel tersebut (`post_id` dan `tag_id`).

3. Apa fungsi attach dan detach pada Filament?

Dalam konteks Relationship Manager di Filament (untuk relasi Many-to-Many):

- Attach: Berfungsi untuk menghubungkan data yang sudah ada. Fitur ini tidak membuat data baru di database utama, melainkan hanya menambahkan baris pemetaan baru ke dalam pivot table. (Contoh: Mengaitkan tag "Laravel" yang sudah ada sebelumnya ke Post baru).
- Detach: Berfungsi untuk memutuskan koneksi/relasi. Fitur ini akan menghapus baris dari pivot table, namun tidak akan menghapus data utama dari database. (Contoh: Melepas tag "Laravel" dari sebuah Post, namun tag "Laravel" tersebut tetap ada dan bisa digunakan oleh Post lain).

4. Mengapa JSON column kurang baik untuk relasi?

Menyimpan ID relasi atau array string di dalam kolom JSON (seperti yang sebelumnya digunakan) memiliki banyak kerugian dibanding tabel relasional yang benar:

- Tidak Ada Integritas Data (Referential Integrity): Kolom JSON tidak bisa dipasangkan fitur keamanan Foreign Key Constraints (misal: Cascade on Delete). Jika Tag bernama "Laravel" dihapus dari master data, nama tersebut masih akan "nyangkut" dan tersimpan di JSON Post.
- Query yang Lambat: Membaca, memfilter, atau mencari data post berdasarkan tag spesifik yang ada di dalam struktur array JSON akan jauh lebih lambat dan memakan banyak memori prosesor database dibandingkan mencari index tabel menggunakan JOIN.
- Redudansi: Jika ada perubahan struktur atau nama tag, kita harus melakukan iterasi pembaruan ke setiap baris JSON di tabel posts secara manual.